Taquillero: objetos de base de datos para quitar las consultas directas de los DAO

Zaca - Terminal de autobuses | 27 de octubre de 2026 | Alcance: la pantalla Venta del taquillero (paneles Destino, Horario, Asientos y Titular y tarifas) y el paso que sigue, confirmar la venta; contrastado con la base real **terminal** (PostgreSQL 16.9).

1. Resumen

Los DAO que usa la pantalla Venta son cuatro y **ninguno llama todavia a un objeto que exista en la base**: **HorarioDAO** llama a una funcion que aun no esta creada, y **AsientoDAO**, **TarifaDAO** y **TerminalDAO** arman SQL de tablas dentro de Java (4 consultas). Ademas, el paso que falta (confirmar e imprimir) escribe en 4 tablas y por la regla del proyecto debe ser un **procedure**. Regla del contrato: **vista** para consultas armadas, **funcion** para lecturas con parametros, **procedure** para lo que modifica datos, **trigger** para validar.

Concepto	Cantidad	Detalle
Metodos de DAO revisados	6	HorarioDAO 1, AsientoDAO 2, TarifaDAO 1, TerminalDAO 2 (los dos que usa Venta; el resto de TerminalDAO es del administrador).
Vistas nuevas	2	`administracion.v_terminales` (compartida con el documento del administrador) y `ventas.v_tarifas`.
Funciones nuevas	3	`viajes.fn_destinos_validos`, `viajes.fn_asientos_ocupados` y `viajes.fn_buscar_salidas` (esta ya estaba escrita en DAOS-TAQUILLERO.txt; aqui se le agregan 2 columnas).
Procedure nuevo	1	`ventas.sp_registrar_venta`: crea el pasajero, la venta con su folio y los boletos en una sola transaccion. Aun no hay Java que lo llame.
Metodos que desaparecen	1	`AsientoDAO.autobus()`: con las 2 columnas nuevas, `fn_buscar_salidas` ya trae el acomodo y la capacidad del autobus.
Triggers nuevos	0	Ya existen los que validan la venta (tramo, capacidad, folio). Hay decisiones abiertas sobre reglas que faltan (seccion 8).

Convenciones

Las mismas de los documentos del administrador y del despachador: vistas **v_*****, funciones **fn_*****, procedures **sp_*****. Los errores de regla salen con **RAISE EXCEPTION ... USING ERRCODE** y Java los lee con `e.getSQLState()`. Los codigos nuevos se marcan *(nuevo)* y faltan por confirmar. Cada DAO lee el ResultSet **por posicion**, asi que las columnas deben salir en el orden que indican las tablas.

2. Inventario de los DAO de la pantalla Venta

DAO	Metodo	Consulta de hoy	Se reemplaza por	Estado en la base
TerminalDAO	listar()	SELECT id_terminal, nombre_terminal FROM administracion.terminal ORDER BY nombre_terminal	`administracion.v_terminales`	**NUEVA** (la comparte el administrador)
TerminalDAO	obtenerDestinosValidos(origen)	SELECT DISTINCT con ruta_paradas dos veces + precio_tramo + terminal	`viajes.fn_destinos_validos`	**NUEVA**
HorarioDAO	buscar(origen, destino, fecha)	SELECT ... FROM viajes.fn_buscar_salidas(?, ?, ?)	`viajes.fn_buscar_salidas`	Escrita en DAOS-TAQUILLERO.txt, **no existe en la base**; +2 columnas
AsientoDAO	autobus(idViaje)	SELECT layout + capacidad (viaje, unidad, tipo_bus, layout)	Desaparece: lo trae `fn_buscar_salidas`	Se borra el metodo
AsientoDAO	ocupados(viaje, origen, destino)	SELECT num_asiento de boleto con subconsulta de tramo y int4range	`ventas.fn_asientos_ocupados`	**NUEVA**
TarifaDAO	listar()	SELECT id_tarifa, nombre_tarifa, descuento FROM ventas.tarifa	`ventas.v_tarifas`	**NUEVA**
(sin DAO todavia)	Confirmar e imprimir	INSERT pasajero + UPDATE taquilla (folio) + INSERT venta + INSERT boleto por asiento	`ventas.sp_registrar_venta`	**NUEVO** (seccion 6)

Fuera de este documento: las pantallas Boletos vendidos, Cancelar boleto, Cambiar boleto, Abrir turno y Mostrar al cliente existen como vistas de muestra y todavia no tienen DAO. Cuando se hagan funcionales necesitaran sus propios objetos (por ejemplo cancelar o reemplazar una venta, que usan `cancelada_por` y `reemplaza_a`). Tampoco se incluye `TerminalDAO.crear()`: es del administrador.

3. Lo que se encontro en la base real

Se consulto el esquema vivo (tablas, constraints, vistas, funciones, triggers y datos). Esto precisa o corrige lo que se asumia.

Hallazgo	Efecto en el diseno
fn_buscar_salidas no existe en la base	HorarioDAO ya la llama, asi que el panel Horario falla con 'function does not exist' hasta que se cree. Tampoco existe ninguna de las otras vistas o funciones de este documento.
La taquilla ya tiene un contador de folios	**administracion.taquilla.folios_usados** (default 0) existe en la base. El siguiente folio es **rango_inicial + folios_usados**, sin buscar el maximo en venta. El procedure incrementa el contador en la misma transaccion; si la venta falla, el contador vuelve atras y no se desperdicia el folio (comprobado, seccion 7).
No existe una tabla de turnos	El contrato hablaba de *turno* y existe la pantalla Abrir turno, pero el script y la base no tienen esa tabla. La taquilla de un taquillero sale de **personal_app.id_taquilla** (su cuenta), por eso `sp_registrar_venta` recibe la cuenta (id_app) y de ahi toma la taquilla. Ver decision 1.
Ya hay triggers que validan la venta	`tigresito_preparar_boleto` (BEFORE INSERT/UPDATE en boleto) calcula `tramo_ocupacion` desde las terminales y lanza Z0013 (asiento mayor a la capacidad), Z0014 / Z0015 (terminal fuera de la ruta) y Z0016 (destino no es posterior al origen). `tigresito_validar_folio` lanza Z0002 si el folio esta fuera del rango de la taquilla. **El procedure no repite esas validaciones.**
La sobreventa ya esta protegida	El constraint `evitar_sobreventa` (EXCLUDE gist sobre viaje, asiento y tramo, ignorando boletos cancelados) rechaza un asiento vendido dos veces aunque dos taquillas lo intenten al mismo tiempo. Llega a Java como SQLState **23P01**.
Constraints que ya cubren la venta	`check_recibido` (recibido >= total_pagado), `check_curp_valido` (sexo H, M o X), `unique_folio` (taquilla, folio), `pasajero_curp_key` (CURP unica, puede ser NULL) y `check_descuento` (0 a 1). Los dos primeros llegan como SQLState 23514 y se distinguen por el nombre del constraint.
El precio no lo manda Java	El contrato dice que el total del front no se confia. `sp_registrar_venta` calcula cada precio como **round(precio del tramo x (1 - descuento), 2)** y el total como la suma. El front solo lo muestra, con la misma formula.
El pasajero no tiene columna de identificacion	Solo existe `curp` (opcional y unica). Con CURP se reutiliza el pasajero si ya existe; sin CURP se crea uno nuevo en cada venta. Ver decision 4.
Los ids son GENERATED ALWAYS AS IDENTITY	El INSERT no manda id; el procedure devuelve el id de la venta y el folio impreso por dos parametros **INOUT** (mismo patron que `sp_guardar_ruta`).
La aplicacion se conecta como **postgres**	Igual que en los otros documentos. Con el rol de la aplicacion (solo SELECT en vistas y EXECUTE en funciones y procedures), `sp_registrar_venta` debe ser **SECURITY DEFINER** con SET search_path fijo porque escribe en 4 tablas.
Datos hoy	4 tarifas (Adulto 0, Estudiante 0.5, INAPAM 0.5, Infantil 0.5), 4 terminales (Obregon y cacahoatan no estan en ninguna ruta), 4 taquillas, 1 cuenta de taquillero, 1 ruta con 2 paradas y 1 precio (180.00), 0 ventas y 0 viajes. Para ver horarios hace falta programar un viaje futuro en esa ruta.

4. Vistas (2)

Los DAO leen el ResultSet por posicion. Una vista no garantiza el orden de las filas: cada DAO conserva su ORDER BY al hacer el SELECT.

`administracion.v_terminales (NUEVA, compartida con el administrador)`

Combo Origen de Venta. Reemplaza `TerminalDAO.listar()`. El DAO hace `SELECT id_terminal, nombre_terminal FROM administracion.v_terminales ORDER BY nombre_terminal`. Es la misma vista que ya pedia el documento del administrador; se crea una sola vez.

Pos	Columna	Tipo	NULL	Como la lee Java
1	`id_terminal`	integer	NO	getInt(1) -> Catalogo.id
2	`nombre_terminal`	text	NO	getString(2) -> Catalogo.nombre

`ventas.v_tarifas (NUEVA)`

Tarifas de cada asiento. Reemplaza `TarifaDAO.listar()`. El DAO hace `SELECT id_tarifa, nombre_tarifa, descuento FROM ventas.v_tarifas ORDER BY id_tarifa`. La primera tarifa de la lista es la que se pone por omision a cada asiento nuevo (hoy Adulto, id 1).

Pos	Columna	Tipo	NULL	Como la lee Java
1	`id_tarifa`	integer	NO	getInt(1) -> Tarifa.id
2	`nombre_tarifa`	text	NO	getString(2) -> Tarifa.nombre (la pantalla la muestra con inicial mayuscula)
3	`descuento`	numeric(4,3)	NO	getBigDecimal(3) -> Tarifa.descuento. 0.500 es 50 %

5. Funciones (3): columnas exactas

`viajes.fn_destinos_validos(p_id_origen INT) (NUEVA)`

Combo Destino, que se vuelve a llenar cada vez que cambia el origen. Reemplaza `TerminalDAO.obtenerDestinosValidos(idOrigen)`. Llamada: `SELECT id_terminal, nombre_terminal FROM viajes.fn_destinos_validos(?)` con `ps.setInt(1, idOrigen)`. Devuelve las terminales posteriores a la del origen en alguna ruta y cuyo tramo tiene precio; sin repetidas, ordenadas por nombre. Desde una terminal sin rutas devuelve 0 filas.

Pos	Columna	Tipo	NULL	Como la lee Java
1	`id_terminal`	integer	NO	getInt(1) -> Catalogo.id
2	`nombre_terminal`	text	NO	getString(2) -> Catalogo.nombre

`viajes.fn_buscar_salidas(p_id_origen INT, p_id_destino INT, p_fecha DATE) (ya escrita; +2 columnas)`

Panel Horario. Es la funcion de DAOS-TAQUILLERO.txt con **dos columnas nuevas al final** (acomodo y capacidad), que sustituyen a `AsientoDAO.autobus()`. Llamada: `SELECT id_viaje, fecha_salida, tipo_bus, servicio, placa, libres, precio, acomodo, capacidad FROM viajes.fn_buscar_salidas(?, ?, ?)`. Solo viajes PROGRAMADO o ABORDANDO que aun no salen, de esa fecha en hora de Mexico, cuya ruta pasa por el origen antes que por el destino. Las 7 primeras columnas no cambian.

Pos	Columna	Tipo	NULL	Como la lee Java
1	`id_viaje`	integer	NO	getInt(1) -> Horario.idViaje
2	`fecha_salida`	timestamptz	NO	getObject(2, OffsetDateTime.class).atZoneSameInstant(Formato.ZONA) -> Horario.fecha
3	`tipo_bus`	text	NO	getString(3) -> Horario.tipoBus (PRIMERA, EJECUTIVO...)
4	`servicio`	text	NO	getString(4) -> Horario.servicio
5	`placa`	text	NO	getString(5) -> Horario.placa
6	`libres`	integer	NO	getInt(6) -> Horario.libres: capacidad menos boletos no cancelados que cruzan el tramo
7	`precio`	numeric	NO	getBigDecimal(7) -> Horario.precio (precio del tramo, sin descuento)
8	`acomodo (nueva)`	varchar(3)	NO	getString(8) -> nuevo campo del registro Horario; es el layout, por ejemplo 2+2. Sustituye a Autobus.acomodo
9	`capacidad (nueva)`	integer	NO	getInt(9) -> nuevo campo de Horario. Sustituye a Autobus.capacidad

`ventas.fn_asientos_ocupados(p_id_viaje INT, p_id_origen INT, p_id_destino INT) (NUEVA)`

Panel Asientos: los que salen deshabilitados. Reemplaza `AsientoDAO.ocupados(...)`. Llamada: `SELECT num_asiento FROM ventas.fn_asientos_ocupados(?, ?, ?)`. Un boleto cancelado libera su asiento; quien baja en una parada no choca con quien sube en ella porque el tramo es semiabierto [origen, destino). Ordenado por asiento.

Pos	Columna	Tipo	NULL	Como la lee Java
1	`num_asiento`	integer	NO	getInt(1) -> el DAO lo agrega al Set de ocupados

6. Procedure (1): confirmar la venta

`ventas.sp_registrar_venta (NUEVO)`

Es el paso que falta de la pantalla Venta (boton Confirmar e imprimir). **LANGUAGE plpgsql**; todo ocurre en una sola transaccion: si algo falla no queda ni el pasajero, ni el folio gastado, ni boletos a medias. El procedure devuelve una fila con `p_id_venta` y `p_folio` (por ejemplo TAP1-000001, el mismo texto que ya arman las vistas de abordaje y manifiesto).

#	Parametro	Tipo	NULL	Valor que manda Java
1	`p_id_venta` (INOUT)	integer	SI	Entra NULL; sale el id de la venta nueva
2	`p_folio` (INOUT)	text	SI	Entra NULL; sale el folio impreso, serie-000000, para el ticket y el QR
3	`p_id_vendedor`	integer	NO	Id de la cuenta en sesion (personal_app.id_app). De aqui sale la taquilla: no se manda aparte para que nadie venda en una taquilla ajena
4	`p_id_viaje`	integer	NO	Horario elegido: idViaje()
5	`p_id_origen`	integer	NO	cbOrigen.getValue().id()
6	`p_id_destino`	integer	NO	cbDestino.getValue().id()
7	`p_nombre`	text	NO	txtNombre (el formulario ya lo exige)
8	`p_ap_paterno`	text	NO	txtApPaterno
9	`p_ap_materno`	text	NO	txtApMaterno (en la tabla es NOT NULL)
10	`p_curp`	text	SI	En mayusculas, o NULL si el campo va vacio (nunca cadena vacia). El CHECK exige ^[A-Z]{4}[0-9]{6}[HMX][A-Z]{5}[A-Z0-9][0-9]$
11	`p_asientos`	jsonb	NO	Arreglo con un objeto por asiento: [{"num":3,"id_tarifa":1},{"num":4,"id_tarifa":2}]. Java: setString + marcador ?::jsonb
12	`p_recibido`	numeric	NO	Efectivo recibido. Debe ser mayor o igual al total que calcula la base

Que hace, en orden

Paso	Accion	Si falla
1	Revisa que haya al menos un asiento.	`Z0020` (nuevo)
2	Toma la taquilla activa de la cuenta y gasta un folio: `folios_usados = folios_usados + 1`; folio = rango_inicial + folios_usados - 1.	`Z0018` (nuevo): la cuenta no tiene taquilla activa
3	Busca el precio del tramo origen-destino del viaje.	`Z0019` (nuevo): el viaje no recorre ese tramo o no tiene precio
4	Calcula el total: suma de round(precio x (1 - descuento), 2) por asiento.	-
5	Pasajero: con CURP lo busca y lo reutiliza (INSERT ... ON CONFLICT (curp)); sin CURP crea uno nuevo.	`23514` check_curp_valido
6	Inserta la venta (folio, titular, vendedor, total, recibido).	`23514` check_recibido; `Z0002` folio fuera de rango
7	Inserta un boleto por asiento en estado VIGENTE con su precio. El trigger calcula el tramo y valida terminales y capacidad.	`23P01`, `Z0013` a `Z0016`
8	Arma el folio impreso y lo devuelve junto con el id de la venta.	-

Llamada desde Java y errores que llegan

Como el procedure tiene parametros INOUT, el DAO usa `prepareStatement("CALL ventas.sp_registrar_venta(?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?::jsonb, ?)")` con `executeQuery()` y lee `rs.getInt(1)` y `rs.getString(2)`. Los NULL de los INOUT y de la CURP se mandan con `setObject(i, null)`; ese mismo patron funciono con `sp_guardar_ruta`.

SQLState	Cuando	Mensaje sugerido en pantalla
`23P01`	El asiento ya lo vendio otra taquilla en un tramo que se cruza (constraint evitar_sobreventa).	Ese asiento ya se vendio. Actualice los asientos y elija otro.
`23514`	Efectivo menor al total (constraint check_recibido).	El efectivo recibido es menor al total.
`23514`	CURP con formato invalido (constraint check_curp_valido).	La CURP no tiene un formato valido.
`Z0002`	El folio queda fuera del rango de la taquilla: se acabaron los folios.	La taquilla ya no tiene folios disponibles.
`Z0013`	El asiento es mayor a la capacidad de la unidad.	Ese asiento no existe en este autobus.
`Z0014` / `Z0015` / `Z0016`	Terminal fuera de la ruta o destino no posterior al origen. No deberia pasar: los combos salen de fn_destinos_validos.	Mensaje con el texto de la base.
`Z0018` (nuevo)	La cuenta no tiene taquilla activa asignada.	Su cuenta no tiene taquilla asignada. Avise al administrador.
`Z0019` (nuevo)	El viaje no recorre el tramo elegido o el tramo no tiene precio.	Ese horario ya no vende ese tramo.
`Z0020` (nuevo)	La venta no trae asientos.	Elija al menos un asiento.

Dos errores comparten el SQLState 23514; el DAO los distingue con el nombre del constraint (`e.getServerErrorMessage().getConstraint()`).

7. Pruebas realizadas contra la base real

Se crearon los 6 objetos dentro de una transaccion (BEGIN ... ROLLBACK) con datos de mentira: un conductor con licencia vigente y un viaje de manana a las 10:00 en la ruta Pijijiapan - Tapachula. Al final se comprobo que no quedo nada: 0 ventas, 0 viajes, 0 folios usados y ningun objeto nuevo.

Prueba	Resultado	Conclusion
Crear 2 vistas, 3 funciones y el procedure	Todo se creo sin error	La sintaxis y los tipos son validos en PostgreSQL 16.9.
v_terminales y v_tarifas	4 terminales ordenadas por nombre; 4 tarifas con descuento 0, 0.5, 0.5 y 0.5	Coinciden con los catalogos reales.
fn_destinos_validos	Desde Pijijiapan: TAPACHULA. Desde Tapachula: 0 filas	Solo ofrece sentidos validos con precio.
fn_buscar_salidas(12, 1, manana)	1 fila: 10:00, PRIMERA, ORDINARIO, 60 libres, precio 180.00, acomodo 2+2, capacidad 60	Las 9 columnas salen en el orden indicado.
Venta 1: asientos 3 (Adulto) y 4 (Estudiante), recibido 300	Folio TAP1-000001; total 270.00; boletos de 180.00 y 90.00; tramo [1,2); estado 1; folios_usados = 1	Precios con descuento calculados por la base.
Despues de la venta 1	fn_asientos_ocupados devuelve 3 y 4; fn_buscar_salidas muestra 58 libres	Las lecturas ven los boletos recien vendidos.
Venta 2: misma CURP, asiento 10 (INAPAM)	Folio TAP1-000002; sigue habiendo 1 pasajero y ya son 2 ventas	La CURP repetida reutiliza al pasajero.
Venta 3: sin CURP	Folio TAP1-000003; ahora hay 2 pasajeros	Sin CURP se crea un pasajero nuevo.
Asiento 3 otra vez	23P01: viola la restriccion de exclusion evitar_sobreventa	La sobreventa se rechaza.
Efectivo 100 con total 180	23514: viola check_recibido	La base valida el efectivo.
Asiento 99	Z0013: el asiento 99 no existe, la unidad solo tiene 60	El trigger de boletos valida la capacidad.
Tramo inverso (Tapachula a Pijijiapan)	Z0019: el viaje no recorre el tramo o no tiene precio	Error propio del procedure.
Arreglo vacio / cuenta 999 / CURP 'XXXX'	Z0020 / Z0018 / 23514 check_curp_valido	Cada guarda responde con su error.
Folios despues de 3 ventas y 7 intentos fallidos	folios_usados = 3	Un error no gasta folios.
Despues del ROLLBACK	0 ventas, 0 viajes, folios_usados = 0, 0 objetos nuevos	No quedo ningun dato de prueba.

Efecto secundario: las secuencias de identidad de PostgreSQL no se revierten con ROLLBACK, asi que los proximos ids de venta, pasajero, viaje y personal tendran saltos. No afecta nada.

8. Decisiones y discrepancias que el equipo debe resolver

#	Tema	Situacion	Que hay que decidir o hacer
1	Turno y taquilla del taquillero	No hay tabla de turnos en el script ni en la base, pero existe la pantalla Abrir turno y el contrato la describia. Hoy la taquilla sale de la cuenta (`personal_app.id_taquilla`).	Decidir: (a) la taquilla es fija por cuenta y se quita o se deja de muestra Abrir turno, o (b) agregar una tabla de turnos al script y que el procedure tome la taquilla del turno abierto. En (a), ademas, falta una regla que obligue a que toda cuenta TAQUILLERO tenga taquilla (hoy es opcional); el procedure ya protege con Z0018.
2	Vender un viaje que ya salio	Nada en la base impide vender un viaje CERRADO, EN RUTA o con hora pasada: solo lo evita fn_buscar_salidas al listar. Un cliente con la pantalla abierta mucho tiempo podria venderlo.	Decidir si el procedure debe rechazar viajes que no esten PROGRAMADO o ABORDANDO, o cuya salida ya paso, con un codigo nuevo (por ejemplo Z0021). Recomendado: si, es la ultima defensa.
3	Pasajero repetido con otro nombre	Con una CURP que ya existe se reutiliza el pasajero y **no se cambian sus nombres**. Si el taquillero escribio el nombre distinto (o con un error), el ticket impreso lleva el nombre capturado pero la base conserva el anterior.	Decidir si se conserva el nombre guardado (como esta), si se actualiza (cambia tambien los boletos viejos de esa persona) o si el nombre del ticket se guarda en la venta.
4	Identificacion sin CURP	El diseno de la pantalla aceptaba INE u otra identificacion. El script solo tiene `curp`. La pantalla ya se cambio a nombre, apellidos y CURP opcional.	Decidir si basta con CURP opcional (quien no la tenga queda sin identificacion registrada) o si se agrega una columna `identificacion` a pasajero, como proponia el contrato.
5	Redondeo del precio	La base redondea cada asiento a 2 decimales (round, mitades hacia arriba) y suma. La pantalla usa HALF_UP con la misma formula. Hoy los descuentos son 0 y 0.5, asi que no hay diferencias.	Si algun dia hay descuentos con mas decimales, cambiar la formula en un solo lugar y avisar al otro.
6	Unidades en taller	El viaje ya programado puede tener una unidad que despues entre al taller. fn_buscar_salidas lo seguiria mostrando.	Ver la decision 1 del documento del despachador (FDW). Para la venta, decidir si un viaje con unidad no apta debe dejar de aparecer.
7	Permisos y SECURITY DEFINER	La aplicacion usa postgres. `sp_registrar_venta` escribe en pasajero, taquilla, venta y boleto.	Con el rol de la aplicacion debe ser SECURITY DEFINER con search_path fijo, y las vistas y funciones nuevas deben recibir GRANT SELECT / EXECUTE.
8	Codigos de error nuevos	Se proponen Z0018 (cuenta sin taquilla), Z0019 (tramo sin precio) y Z0020 (venta sin asientos). El documento del despachador proponia Z0017 para unidad no apta.	Confirmar los codigos con el equipo y agregarlos al contrato de errores y a Mensaje.

Orden sugerido para implementarlo

Paso	Que se hace	Archivos de Java que cambian
1	Crear en la base las 2 vistas y las 3 funciones (el SQL probado esta en el apendice). Con esto el panel Horario vuelve a funcionar.	Ninguno
2	Cambiar los DAO de lectura: TerminalDAO (listar y destinos), TarifaDAO, AsientoDAO (solo ocupados) y HorarioDAO (leer acomodo y capacidad).	TerminalDAO, TarifaDAO, AsientoDAO, HorarioDAO, Horario.java; se borra Autobus.java
3	Resolver las decisiones 1 a 4 y crear sp_registrar_venta.	Ninguno
4	Hacer funcional Confirmar e imprimir: validar los campos del titular, llamar al procedure y mostrar el ticket con el folio devuelto.	VentaController y un DAO de venta nuevo

Apendice: SQL probado (listo para copiar)

Es el SQL que se ejecuto en la transaccion de prueba, con el estilo del script (mayusculas, INNER JOIN, sin sangria en los bloques sencillos). Si ya creaste `v_terminales` para el administrador, omite la primera vista.

```
CREATE OR REPLACE VIEW administracion.v_terminales AS
SELECT id_terminal, nombre_terminal
FROM administracion.terminal
ORDER BY nombre_terminal;


CREATE OR REPLACE VIEW ventas.v_tarifas AS
SELECT id_tarifa, nombre_tarifa, descuento
FROM ventas.tarifa
ORDER BY id_tarifa;


CREATE OR REPLACE FUNCTION viajes.fn_destinos_validos(p_id_origen INT)
RETURNS TABLE (id_terminal INT, nombre_terminal TEXT)
LANGUAGE sql
AS $$
SELECT DISTINCT t.id_terminal, t.nombre_terminal
FROM viajes.ruta_paradas o
INNER JOIN viajes.ruta_paradas d ON d.id_ruta = o.id_ruta AND d.orden > o.orden
INNER JOIN viajes.precio_tramo p ON p.id_ruta = o.id_ruta AND p.orden_origen = o.orden AND p.orden_destino = d.orden
INNER JOIN administracion.terminal t ON t.id_terminal = d.id_terminal
WHERE o.id_terminal = p_id_origen
ORDER BY t.nombre_terminal;
$$;


CREATE OR REPLACE FUNCTION viajes.fn_buscar_salidas(p_id_origen INT, p_id_destino INT, p_fecha DATE)
RETURNS TABLE (
id_viaje INT, fecha_salida TIMESTAMPTZ, tipo_bus TEXT, servicio TEXT, placa TEXT, libres INT, precio DECIMAL,
acomodo VARCHAR, capacidad INT
)
LANGUAGE sql
AS $$
SELECT
vp.id_viaje, vp.fecha_salida, ntb.nombre_tipo_bus, ts.nombre_servicio, u.placa,
tb.capacidad - (SELECT count(*) FROM ventas.boleto b
                WHERE b.id_viaje = vp.id_viaje AND b.id_estado_boleto <> 4
                AND b.tramo_ocupacion && int4range(o.orden, d.orden, '[)'))::INT,
pt.precio, l.nombre_layout, tb.capacidad
FROM viajes.viaje_programado vp
INNER JOIN viajes.estado_viaje ev ON ev.id_estado_viaje = vp.id_estado_viaje
INNER JOIN viajes.ruta r ON r.id_ruta = vp.id_ruta
INNER JOIN viajes.tipo_servicio ts ON ts.id_servicio = r.id_servicio
INNER JOIN viajes.unidad u ON u.id_unidad = vp.id_unidad
INNER JOIN viajes.tipo_bus tb ON tb.id_tipo_bus = u.id_tipo_bus
INNER JOIN viajes.nombre_tipo_bus ntb ON ntb.id_nombre_tipo_bus = tb.id_nombre_tipo_bus
INNER JOIN viajes.layout l ON l.id_layout = tb.id_layout
INNER JOIN viajes.ruta_paradas o ON o.id_ruta = vp.id_ruta AND o.id_terminal = p_id_origen
INNER JOIN viajes.ruta_paradas d ON d.id_ruta = vp.id_ruta AND d.id_terminal = p_id_destino AND d.orden > o.orden
INNER JOIN viajes.precio_tramo pt ON pt.id_ruta = vp.id_ruta AND pt.orden_origen = o.orden AND pt.orden_destino = d.orden
```

```
WHERE ev.nombre_estado_viaje IN ('PROGRAMADO', 'ABORDANDO')
AND vp.fecha_salida > now()
AND (vp.fecha_salida AT TIME ZONE 'America/Mexico_City')::date = p_fecha
ORDER BY vp.fecha_salida;
$$;


CREATE OR REPLACE FUNCTION ventas.fn_asientos_ocupados(p_id_viaje INT, p_id_origen INT, p_id_destino INT)
RETURNS TABLE (num_asiento INT)
LANGUAGE sql
AS $$
SELECT b.num_asiento
FROM ventas.boleto b
WHERE b.id_viaje = p_id_viaje
AND b.id_estado_boleto <> 4
AND b.tramo_ocupacion && (
    SELECT int4range(o.orden, d.orden, '[)')
    FROM viajes.viaje_programado vp
    INNER JOIN viajes.ruta_paradas o ON o.id_ruta = vp.id_ruta AND o.id_terminal = p_id_origen
    INNER JOIN viajes.ruta_paradas d ON d.id_ruta = vp.id_ruta AND d.id_terminal = p_id_destino
    WHERE vp.id_viaje = p_id_viaje)
ORDER BY b.num_asiento;
$$;


CREATE OR REPLACE PROCEDURE ventas.sp_registrar_venta(
INOUT p_id_venta INT, INOUT p_folio TEXT,
p_id_vendedor INT, p_id_viaje INT, p_id_origen INT, p_id_destino INT,
p_nombre TEXT, p_ap_paterno TEXT, p_ap_materno TEXT, p_curp TEXT,
p_asientos JSONB, p_recibido DECIMAL
)
LANGUAGE plpgsql
AS $$
DECLARE
    v_id_taquilla INT;
    v_serie TEXT;
    v_folio INT;
    v_id_titular INT;
    v_precio_base DECIMAL;
    v_total DECIMAL;
BEGIN
    IF p_asientos IS NULL OR jsonb_array_length(p_asientos) = 0 THEN
        RAISE EXCEPTION 'La venta debe tener al menos un asiento.' USING ERRCODE = 'Z0020';
    END IF;

    UPDATE administracion.taquilla q
    SET folios_usados = q.folios_usados + 1
    FROM administracion.personal_app pa
    WHERE pa.id_app = p_id_vendedor AND q.id_taquilla = pa.id_taquilla AND q.activo
    RETURNING q.id_taquilla, q.serie_folio, q.rango_inicial + q.folios_usados - 1
    INTO v_id_taquilla, v_serie, v_folio;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'La cuenta no tiene una taquilla activa asignada.' USING ERRCODE = 'Z0018';
    END IF;

    SELECT pt.precio INTO v_precio_base
```

```
    FROM viajes.viaje_programado vp
    INNER JOIN viajes.ruta_paradas o ON o.id_ruta = vp.id_ruta AND o.id_terminal = p_id_origen
    INNER JOIN viajes.ruta_paradas d ON d.id_ruta = vp.id_ruta AND d.id_terminal = p_id_destino AND d.orden > o.orden
    INNER JOIN viajes.precio_tramo pt ON pt.id_ruta = vp.id_ruta AND pt.orden_origen = o.orden AND pt.orden_destino = d.orden
    WHERE vp.id_viaje = p_id_viaje;
    IF NOT FOUND THEN
        RAISE EXCEPTION 'Ese viaje no recorre el tramo elegido o el tramo no tiene precio.' USING ERRCODE = 'Z0019';
    END IF;

    SELECT sum(round(v_precio_base * (1 - t.descuento), 2)) INTO v_total
    FROM jsonb_to_recordset(p_asientos) AS a(num INT, id_tarifa INT)
    INNER JOIN ventas.tarifa t ON t.id_tarifa = a.id_tarifa;

    IF p_curp IS NULL THEN
        INSERT INTO ventas.pasajero (nombre_pasajero, ap_paterno, ap_materno)
        VALUES (p_nombre, p_ap_paterno, p_ap_materno)
        RETURNING id_pasajero INTO v_id_titular;
    ELSE
        INSERT INTO ventas.pasajero (nombre_pasajero, ap_paterno, ap_materno, curp)
        VALUES (p_nombre, p_ap_paterno, p_ap_materno, p_curp)
        ON CONFLICT (curp) DO UPDATE SET curp = EXCLUDED.curp
        RETURNING id_pasajero INTO v_id_titular;
    END IF;

    INSERT INTO ventas.venta (id_taquilla, folio, id_titular, id_vendedor, total_pagado, recibido)
    VALUES (v_id_taquilla, v_folio, v_id_titular, p_id_vendedor, v_total, p_recibido)
    RETURNING id_venta INTO p_id_venta;

    INSERT INTO ventas.boleto (id_venta, id_viaje, id_terminal_origen, id_terminal_destino, num_asiento,
    id_tarifa, id_estado_boleto, precio_unitario)
    SELECT p_id_venta, p_id_viaje, p_id_origen, p_id_destino, a.num, a.id_tarifa,
    (SELECT id_estado_boleto FROM ventas.estado_boleto WHERE nombre_estado_boleto = 'VIGENTE'),
    round(v_precio_base * (1 - t.descuento), 2)
    FROM jsonb_to_recordset(p_asientos) AS a(num INT, id_tarifa INT)
    INNER JOIN ventas.tarifa t ON t.id_tarifa = a.id_tarifa;

    p_folio := upper(v_serie || '-' || lpad(v_folio::text, 6, '0'));
END;
$$;
```